

Scientific Conference Organization Tool

Grupo 7:

- Tomás Esteves - Up202205045
- Elton Amado Vaz - Up202309925
- Pedro Dinis – 202205947

Content

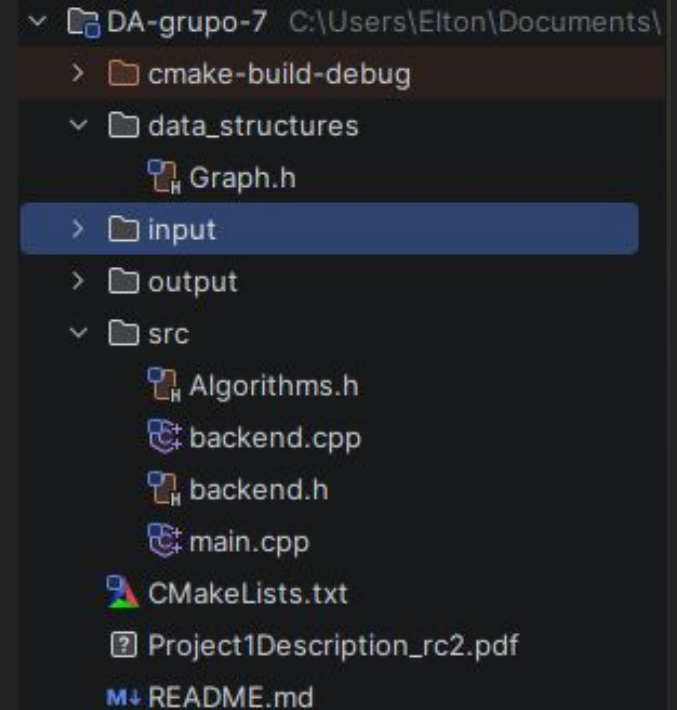
1. O Desafio: Atribuição Inteligente de Revisões
2. Organização do Projeto
3. Modelagem do Problema: O Grafo
4. Motor de Atribuição: Edmonds-Karp
5. Análise de Risco($K=1$ e $K>1$)
6. Interface: Interatividade
7. Conclusão

O Desafio: Atribuição Inteligente de Revisões

- **Objetivo Principal:** Atribuir artigos científicos a revisores qualificados de forma automática e otimizada.
- **Restrições do Sistema:**
 - **Artigos:** Exigem um número mínimo de revisões (MinReviews).
 - **Revisores:** Têm um limite máximo de carga de trabalho (MaxReviews).
- **Regras de Compatibilidade:** Cruzamento de Domínios e Especialidades (Primárias e Secundárias).
- **O Fator Crítico:** Garantir a resiliência da conferência perante a ausência inesperada de revisores (Análise de Risco).

Organização do Projeto

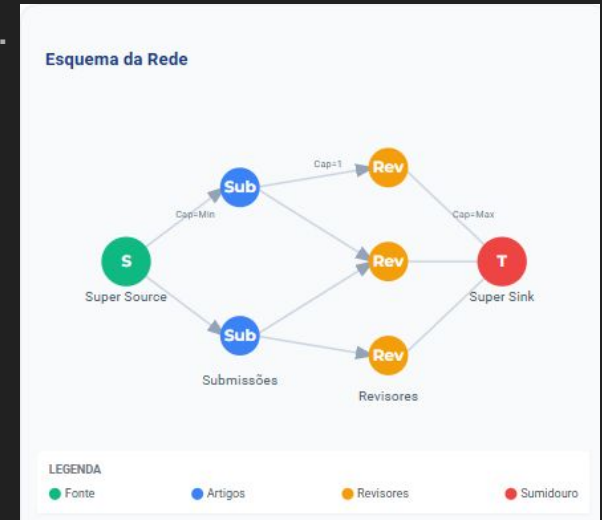
- Graph.h
- Algorithms.h
- backend.cpp
- backend.h
- main.cpp



Modelagem do Problema: O Grafo

Rede de Fluxo Direcionada

- Super Source (S): Nó artificial que alimenta a rede.
 - Liga a todas as Submissões.
 - Capacidade = $\text{MinReviewsPerSubmission}$.
- Nós Intermediários: Arestas de decisão.
 - Ligação: Submissão $\rightarrow$ Revisor.
 - Capacidade = 1.
- Super Sink (T): Nó artificial de destino.
 - Recebe arestas de todos os Revisores.
 - Capacidade = $\text{MaxReviewsPerReviewer}$.



Motor de Atribuição: Edmonds-Karp

Complexidade Temporal

- testAndVisit(): $O(1)$
- findAugmentingPath(): $O(V+E)$
- findMinResidualAlongPath(): $O(V)$
- augmentFlowAlongPath(): $O(V)$
- edmondsKarp(): $O(V * E^2)$

```
template <class T>
void testAndVisit(std::queue<Vertex<T>*> &q, Edge<T> *e, Vertex<T> *w, double residual){...}

template <class T>
bool findAugmentingPath(Graph<T> *g, Vertex<T> *s, Vertex<T> *t){...}

template <class T>
double findMinResidualAlongPath(Vertex<T> *s, Vertex<T> *t){...}

template <class T>
void augmentFlowAlongPath(Vertex<T> *s, Vertex<T> *t, double f){...}

template <class T>
void edmondsKarp(Graph<T> *g, T sourceInfo, T targetInfo){...}
```

Análise de Risco(K=1 e K>1)

➤ Objetivo e Cenários:

- Identificar revisores cuja ausência compromete a conferência.
- $K = 1$: Falha simples (um revisor adoece).
- $K > 1$: Falhas múltiplas (grupos de pesquisa indisponíveis).

➤ Abordagem Algorítmica:

- 1. **Corte**: Remover arestas Revisor(es) $\rightarrow$ Sink.
- 2. **Reset**: Zerar fluxos atuais de toda a rede.
- 3. **Recálculo**: Executar algoritmo Edmonds-Karp.
- 4. **Verificação**: Fluxo Total < Fluxo Ideal?

➤ Complexidade para K revisores:

- $O((R/K) * V * E^2)$

Interface: Interatividade

1. Load Dataset (.csv): Carrega os dados para a memória e constrói a rede.
2. View/Edit Parameters: Permite consultar as regras lidas do ficheiro (ex: limites de revisões).
3. Run Max-Flow Assignment: Executa o algoritmo de Edmonds-Karp.
4. Risk Analysis (R1/RK): Testa a resiliência da rede contra faltas de revisores.
5. Export Results: Guarda as atribuições e o caminho de fluxo num ficheiro e na consola.
6. Run Batch Mode: Permite correr tudo automaticamente de uma só vez.
- 0 Exit: Fecha o programa.

Thanks For Watching